Ex 1. Perceptron Learning Algorithm

Objective:

To implement the Perceptron Learning Algorithm to classify linearly separable data using the step activation function.

Algorithm:

Step 1: Define the Step Activation Function

- Use the step function to determine the class label:

$$\text{step}(s) = \begin{cases} 1, & \text{if } s \geq 0 \\ 0, & \text{if } s < 0 \end{cases}$$

Step 2: Initialize Parameters

- Set the learning rate (l) to 1.
- Initialize the weight vector w and bias b to zeros.

Step 3: Iterative Training

- For a fixed number of epochs (e.g., 100):
  - For each training example x[i]:
    1. Compute the weighted sum:

$$\text{sum} = w \cdot x[i] + b$$

    2. Compute the predicted label (y_{pred}) using the step function:

$$y_{\text{pred}} = \text{step}(\text{sum})$$

    3. Compute the error (err)

$$\text{err} = y[i] - y_{\text{pred}}$$

    4. Update weights and bias if there is an error:

$$w = w + l \cdot \text{err} \cdot x[i]$$

$$b = b + l \cdot \text{err}$$

Step 4: Final Model Parameters

- After training, return the updated weights w and bias b.

Step 5: Prediction

- For each input example, compute the predicted label using the trained weights and bias:

$$\text{sum} = w \cdot x[i] + b$$

$$y_{\text{pred}} = \text{step}(\text{sum})$$

Program:

```
# Import necessary libraries
import numpy as np
```

```python
# Define the step activation function
def step(s):
    return 1 if s >= 0 else 0

# Define the perceptron training algorithm
def perceptron(x, y):

    # Initialize learning rate
    learning_rate = 1

    # Initialize weights and bias
    w = np.zeros(x.shape[1])  # Weight vector
    b = np.array(0)           # Bias term

    # Perceptron training loop
    for epoch in range(100):  # Maximum number of epochs
        for i in range(x.shape[0]):
            # Calculate the weighted sum
            weighted_sum = np.dot(w, x[i]) + b

            # Predict the output using the step activation function
            y_pred = step(weighted_sum)

            # Calculate the error
            error = y[i] - y_pred

            # Update weights and bias
            w = w + learning_rate * error * x[i]
            b = b + learning_rate * error

    # Print final weights and bias
    print(f"Final Weights: {w}, Final Bias: {b}")

    return w, b

# Define input dataset and target labels
x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  # Input features
y = np.array([0, 1, 1, 1])                      # Target labels

# Train the perceptron model
w, b = perceptron(x, y)

# Evaluate the perceptron model on the input dataset
print("\nInput, Predicted, Actual:")
for i in range(x.shape[0]):
    # Calculate the weighted sum
    weighted_sum = np.dot(w, x[i]) + b
```

```
    # Predict the output using the step activation function
    y_pred = step(weighted_sum)

    # Print the input, predicted output, and actual target
    print(f"Input: {x[i]}, Predicted: {y_pred}, Actual: {y[i]}")
```

Output:

```
Final Weights: [1. 1.], Final Bias: -1

Input, Predicted, Actual:
Input: [0 0], Predicted: 0, Actual: 0
Input: [0 1], Predicted: 1, Actual: 1
Input: [1 0], Predicted: 1, Actual: 1
Input: [1 1], Predicted: 1, Actual: 1
```

Ex 2. Gradient descent algorithm

Objective:

To implement and visualize the gradient descent algorithm for minimizing the quadratic function $y=x^2$.

Algorithm:

Step 1: Define the Function

- Define the quadratic function to minimize:

$$f(x) = x^2$$

Step 2: Set Gradient Descent Parameters

- Initialize:
  - **Learning rate (α)**: Determines the step size in each iteration.
  - **Starting point ($x_{current}$)**: The initial value of x.
  - **Number of iterations**: The number of gradient descent steps.

Step 3: Initialize Storage for Visualization

- Create lists to store x-values and their corresponding function values for each step of the descent.

Step 4: Iterative Gradient Descent

1. Compute the gradient of the function:

$$\text{gradient} = f'(x) = 2x$$

2. Update the value of x using the gradient descent rule:

$$x_{\text{new}} = x_{\text{current}} - \alpha \cdot \text{gradient}$$

3. Update $x_{current}$ with x_{new}.
4. Store the new x-value and corresponding function value for plotting.

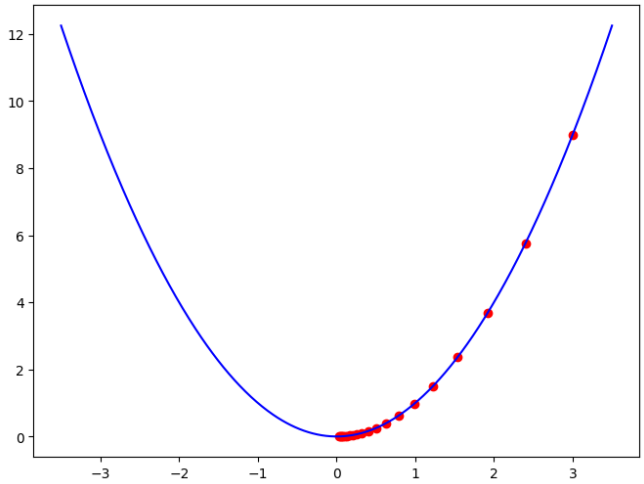
Program:

```
import matplotlib.pyplot as plt
import numpy as np
# Function definition
def function(x):
    return x**2
# Gradient descent parameters
learning_rate = 0.1
x_current = 3  # Starting point
iterations = 20
# Store values for plotting
x_values = [x_current]
y_values = [function(x_current)]
```

```python
# Perform gradient descent
for _ in range(iterations):
    gradient = 2 * x_current  # Derivative of y = x^2
    x_new = x_current - learning_rate * gradient  # Update rule
    x_current = x_new
        # Store the new values
    x_values.append(x_current)
    y_values.append(function(x_current))
# Generate x-axis values for the function
x = np.linspace(-3.5, 3.5, 500)
y = function(x)

# Print the optimization result
print(f"Optimization Result:")
print(f"Optimal x: {x_current:.5f}")
print(f"Minimum value of the function (y): {function(x_current):.5f}")

# Plot the function
plt.figure(figsize=(8, 6))
plt.plot(x, y, label="y = x^2", color="blue")
plt.scatter(x_values, y_values, color="red", label="Gradient Descent Steps")
# Show plot
plt.show()
```

Output:

```
Optimization Result:
Optimal x: 0.03459
Minimum value of the function (y): 0.00120
```



In [32]:
```
import numpy as np
def step(sum):
    if sum>=0:
        return 1
    return 0
def perceptron(x,y):
    l=0.1
    w=np.zeros(x.shape[1])
    b=np.array(0)
    for ep in range(20):
        flag=0
        for i in range(x.shape[0]):
            sum=np.dot(w,x[i])+b
            Y_pred=step(sum)
            e=y[i]-Y_pred
            if e!=0:
                w=w+l*e*x[i]
                b=b+l*e
                flag=1
            print(w,b)
        if flag==0:
            break
    return w,b
```

In [33]:
```
x=np.array([[0,0],[0,1],[1,0],[1,1]])
y=np.array([0,1,1,1])
w,b=perceptron(x,y)
for i in range(x.shape[0]):
    sum=np.dot(w,x[i])+b
    Y_pred=step(sum)
    print(x[i],Y_pred,y[i])
```

```
[0. 0.] -0.1
[0.  0.1] 0.0
[0.  0.1] 0.0
[0.  0.1] 0.0
[0.  0.1] -0.1
[0.  0.1] -0.1
[0.1 0.1] 0.0
[0.1 0.1] 0.0
[0.1 0.1] -0.1
[0.1 0.1] -0.1
[0.1 0.1] -0.1
[0.1 0.1] -0.1
[0.1 0.1] -0.1
[0.1 0.1] -0.1
[0.1 0.1] -0.1
[0.1 0.1] -0.1
[0 0] 0 0
[0 1] 1 1
[1 0] 1 1
[1 1] 1 1
```

In [34]:
```
x=np.array([[0,0],[0,1],[1,0],[1,1]])
y=np.array([0,0,0,1])
w,b=perceptron(x,y)
for i in range(x.shape[0]):
    sum=np.dot(w,x[i])+b
    Y_pred=step(sum)
    print(x[i],Y_pred,y[i])
```

```
[0. 0.] -0.1
[0. 0.] -0.1
[0. 0.] -0.1
[0.1 0.1] 0.0
[0.1 0.1] -0.1
[0.1 0. ] -0.2
[0.1 0. ] -0.2
[0.2 0.1] -0.1
[0.2 0.1] -0.1
[0.2 0. ] -0.2
[0.1 0. ] -0.30000000000000004
[0.2 0.1] -0.20000000000000004
[0.2 0.1] -0.20000000000000004
[0.2 0.1] -0.20000000000000004
[0.2 0.1] -0.20000000000000004
[0.2 0.1] -0.20000000000000004
[0 0] 0 0
[0 1] 0 0
[1 0] 0 0
[1 1] 1 1
```

In [35]:
```
x=np.array([[0,0,0],[0,0,1],[0,1,0],[0,1,1],[1,0,0],[1,0,1],[1,1,0],[1,1,1]])
y=np.array([0,1,1,1,1,1,1,1])
w,b=perceptron(x,y)
for i in range(x.shape[0]):
    sum=np.dot(w,x[i])+b
    Y_pred=step(sum)
    print(x[i],Y_pred,y[i])
```

```
[0. 0. 0.] -0.1
[0.  0.  0.1] 0.0
[0.  0.  0.1] 0.0
[0.  0.  0.1] 0.0
[0.  0.  0.1] 0.0
[0.  0.  0.1] 0.0
[0.  0.  0.1] 0.0
[0.  0.  0.1] 0.0
[0.  0.  0.1] -0.1
[0.  0.  0.1] -0.1
[0.  0.1 0.1] 0.0
[0.  0.1 0.1] 0.0
[0.  0.1 0.1] 0.0
[0.  0.1 0.1] 0.0
[0.  0.1 0.1] 0.0
[0.  0.1 0.1] 0.0
[0.  0.1 0.1] -0.1
[0.  0.1 0.1] -0.1
[0.  0.1 0.1] -0.1
[0.  0.1 0.1] -0.1
[0.1 0.1 0.1] 0.0
[0.1 0.1 0.1] 0.0
[0.1 0.1 0.1] 0.0
[0.1 0.1 0.1] 0.0
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0.1 0.1 0.1] -0.1
[0 0 0] 0 0
[0 0 1] 1 1
[0 1 0] 1 1
[0 1 1] 1 1
[1 0 0] 1 1
[1 0 1] 1 1
[1 1 0] 1 1
[1 1 1] 1 1
```

In [ ]:

jupyter week-3 Last Checkpoint: 2 seconds ago

File Edit View Run Kernel Settings Help

Code

[14]:
```
import numpy as np
import matplotlib.pyplot as plt
def square(x):
    return x**2
learning_rate=0.1
x_current=3
iterations=20
x_values=[x_current]
y_values=[square(x_current)]
for i in range(iterations):
    gradient=2*x_current
    x_new=x_current-learning_rate*gradient
    x_current=x_new
    x_values.append(x_current)
    y_values.append(square(x_current))
x=np.linspace(-3.5,3.5,500)
y=square(x)
plt.figure(figsize=(5,4))
plt.plot(x,y,label='y=x^2',color='blue')
plt.scatter(x_values,y_values,color='red',label='Gradient Descent Steps')
plt.plot(x_values,y_values,color='red',linestyle='--',label='path of Descent')
plt.show()
```

Ex. 4 Tensor Operations in TensorFlow

Objective:

To understand the concept of tensors, their attributes, and perform operations such as creation, arithmetic computations, reshaping, slicing, aggregation, and implement activation functions like ReLU and Sigmoid using TensorFlow.

Description:

1. Tensor Creation

- **tf.constant(value, dtype=None, shape=None)**: Creates a constant tensor.
- **tf.zeros(shape, dtype=tf.float32)**: Creates a tensor filled with zeros.
- **tf.ones(shape, dtype=tf.float32)**: Creates a tensor filled with ones.
- **tf.random.normal(shape, mean=0.0, stddev=1.0)**: Creates a tensor with random normal values.
- **tf.random.uniform(shape, minval=0, maxval=None)**: Creates a tensor with random uniform values.

2. Tensor Attributes

- **tensor.shape**: Returns the shape of the tensor.
- **tf.rank(tensor)**: Returns the rank (number of dimensions) of the tensor.
- **tensor.dtype**: Returns the data type of the tensor.
- **tensor.device**: Indicates where the tensor resides (CPU/GPU).

3. Arithmetic Operations

- **tf.add(a, b)**: Element-wise addition of tensors.
- **tf.subtract(a, b)**: Element-wise subtraction.
- **tf.multiply(a, b)**: Element-wise multiplication.
- **tf.divide(a, b)**: Element-wise division.
- **tf.matmul(a, b)**: Matrix multiplication.

---

4. Reshaping and Slicing

- **tf.reshape(tensor, shape)**: Changes the shape of a tensor while maintaining its elements.
- **tensor[start:stop, start:stop]**: Slices a tensor along specific dimensions.
- **tf.expand_dims(tensor, axis)**: Adds an extra dimension to the tensor.
- **tf.squeeze(tensor, axis=None)**: Removes dimensions of size 1 from the tensor.

5. Aggregation Operations

- **tf.reduce_sum(tensor, axis=None)**: Computes the sum of tensor elements along a dimension.
- **tf.reduce_mean(tensor, axis=None)**: Computes the mean of tensor elements.
- **tf.reduce_max(tensor, axis=None)**: Finds the maximum value in a tensor.
- **tf.reduce_min(tensor, axis=None)**: Finds the minimum value in a tensor.

6. Activation Functions

- **tf.nn.relu(tensor)**: Applies the ReLU activation function (f(x) = max(0,x)).
- **tf.nn.sigmoid(tensor)**: Applies the Sigmoid activation function (f(x)=1/(1+e^−x)).

7. Broadcasting

- **tf.add(a, b)**: Automatically broadcasts smaller tensors to match the shape of larger tensors during addition (and similar operations).

8. Gradient Computation

- **with tf.GradientTape() as tape:**: Records operations to compute gradients.
- **tape.gradient(target, sources)**: Computes the gradient of target with respect to sources.

Program:

```python
import tensorflow as tf

# 1. Creating Tensors
scalar = tf.constant(42)
vector = tf.constant([1, 2, 3])
matrix = tf.constant([[1, 2], [3, 4]])
tensor_3d = tf.constant([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print("1. Created Tensors:")
print("Scalar:", scalar)
print("Vector:", vector)
print("Matrix:\n", matrix)
print("3D Tensor:\n", tensor_3d)

# 2. Tensor Attributes
print("\n2. Tensor Attributes:")
print("Shape of Matrix:", matrix.shape)
print("Rank of 3D Tensor:", tf.rank(tensor_3d))
print("Data Type of Scalar:", scalar.dtype)

# 3. Arithmetic Operations
a = tf.constant([[1, 2], [3, 4]])
b = tf.constant([[5, 6], [7, 8]])

add_result = tf.add(a, b)
mul_result = tf.multiply(a, b)

print("\n3. Arithmetic Operations:")
print("Addition:\n", add_result)
print("Element-wise Multiplication:\n", mul_result)

# 4. Reshaping and Slicing
tensor = tf.constant([1, 2, 3, 4, 5, 6])
reshaped_tensor = tf.reshape(tensor, [2, 3])
sliced_tensor = reshaped_tensor[:, 1:]

print("\n4. Reshaping and Slicing:")
print("Reshaped Tensor:\n", reshaped_tensor)
print("Sliced Tensor (Last two columns):\n", sliced_tensor)

# 5. Aggregation Operations
sum_result = tf.reduce_sum(matrix)
mean_result = tf.reduce_mean(matrix)
max_result = tf.reduce_max(matrix)

print("\n5. Aggregation Operations:")
print("Sum of elements in Matrix:", sum_result)
print("Mean of elements in Matrix:", mean_result)
```

```python
print("Max of elements in Matrix:", max_result)

# 6. Activation Functions
input_tensor = tf.constant([[-1.0, 0.0, 1.0], [2.0, -3.0, 4.0]])

relu_output = tf.nn.relu(input_tensor)
sigmoid_output = tf.nn.sigmoid(input_tensor)

print("\n6. Activation Functions:")
print("ReLU Output:\n", relu_output)
print("Sigmoid Output:\n", sigmoid_output)
```

Expected Output:

```
1. Created Tensors:
Scalar: tf.Tensor(42, shape=(), dtype=int32)
Vector: tf.Tensor([1 2 3], shape=(3,), dtype=int32)
Matrix:
 tf.Tensor(
[[1 2]
 [3 4]], shape=(2, 2), dtype=int32)
3D Tensor:
 tf.Tensor(
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]], shape=(2, 2, 2), dtype=int32)

2. Tensor Attributes:
Shape of Matrix: (2, 2)
Rank of 3D Tensor: tf.Tensor(3, shape=(), dtype=int32)
Data Type of Scalar: <dtype: 'int32'>

3. Arithmetic Operations:
Addition:
 tf.Tensor(
[[ 6  8]
 [10 12]], shape=(2, 2), dtype=int32)
Element-wise Multiplication:
 tf.Tensor(
[[ 5 12]
 [21 32]], shape=(2, 2), dtype=int32)

4. Reshaping and Slicing:
Reshaped Tensor:
 tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)
Sliced Tensor (Last two columns):
 tf.Tensor(
```

```
[[2 3]
 [5 6]], shape=(2, 2), dtype=int32)

5. Aggregation Operations:
Sum of elements in Matrix: tf.Tensor(10, shape=(), dtype=int32)
Mean of elements in Matrix: tf.Tensor(2, shape=(), dtype=int32)
Max of elements in Matrix: tf.Tensor(4, shape=(), dtype=int32)

6. Activation Functions:
ReLU Output:
 tf.Tensor(
[[0. 0. 1.]
 [2. 0. 4.]], shape=(2, 3), dtype=float32)
Sigmoid Output:
 tf.Tensor(
[[0.26894143 0.5        0.7310586 ]
 [0.8807971  0.04742587 0.98201376]], shape=(2, 3), dtype=float32)
```

Ex 5. Digit Recognition Using Convolutional Neural Networks (CNN)

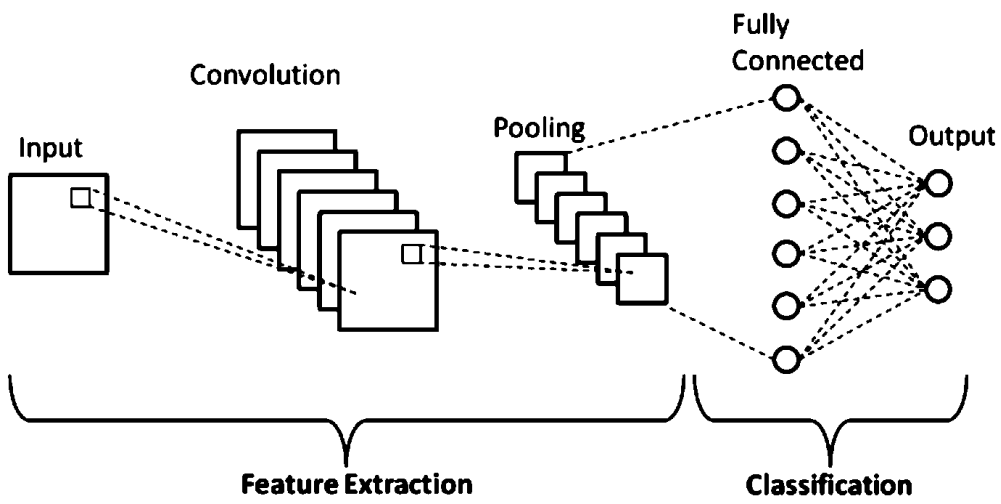
Aim:

To design, implement, and evaluate a Convolutional Neural Network (CNN) model for recognizing handwritten digits from the MNIST dataset

CNN Architecture and Its Layers

1. **Convolutional Layers**: Extract key features like edges, textures, and patterns from the input image by applying filters (kernels).
2. **Pooling Layers**: Reduce the spatial dimensions of feature maps (downsampling) to retain important features while lowering computational complexity.
3. **Flattening Layer**: Converts 2D feature maps into a 1D vector to prepare the data for the fully connected layers.
4. **Fully Connected Layers**: Perform classification by mapping the extracted features to output categories. The last layer uses the softmax activation function for multi-class probability distribution.
5. **Activation Functions**:
    - **ReLU (Rectified Linear Unit)**: Adds non-linearity to the model and helps in faster convergence by avoiding vanishing gradients.
    - **Softmax**: Converts the output of the network into probabilities for each class.
6. **Optimizer**: **Adam** is used for adaptive learning rate optimization, improving training efficiency.
7. **Loss Function**: **Categorical Crossentropy** calculates the difference between actual and predicted labels for multi-class classification problems.

Structure of CNN



Procedure

- **Step 1: Import Libraries** Import necessary libraries such as TensorFlow, NumPy, and Matplotlib for building the model and visualizing data.

- **Step 2: Load and Subset the MNIST Dataset** Load the MNIST dataset and take a subset of 1000 training and 200 test samples to reduce computational complexity.
- **Step 3: Preprocess the Dataset**
    - Normalize the pixel values of the images to the range [0, 1].
    - Reshape the data to add a channel dimension (28x28x1).
    - Convert labels to one-hot encoding using to_categorical.
- **Step 4: Design the CNN Model**
    - Define the CNN model using the Sequential API.
    - Add layers:
        - First Conv2D layer with 4 filters of size (3x3) and ReLU activation.
        - First MaxPooling2D layer with a pool size of (2x2).
        - Second Conv2D layer with 4 filters of size (3x3) and ReLU activation.
        - Second MaxPooling2D layer with a pool size of (2x2).
        - Flatten layer to convert feature maps to a 1D vector.
        - Dense layer with 32 units and ReLU activation.
        - Output Dense layer with 10 units (one for each digit) and Softmax activation.
- **Step 5: Compile the Model** Compile the model using the Adam optimizer, Categorical Crossentropy as the loss function, and accuracy as the evaluation metric.
- **Step 6: Train the Model** Train the model using the training data with a validation split of 20%, 50 epochs, and a batch size of 4.
- **Step 7: Evaluate the Model** Evaluate the trained model on the test dataset and print the accuracy.
- **Step 8: Make Predictions** Select a random image from the test set, predict its class using the model, and compare the predicted label with the actual label.

7. Implementation of Pretrained VGG16 Model for Image Classification

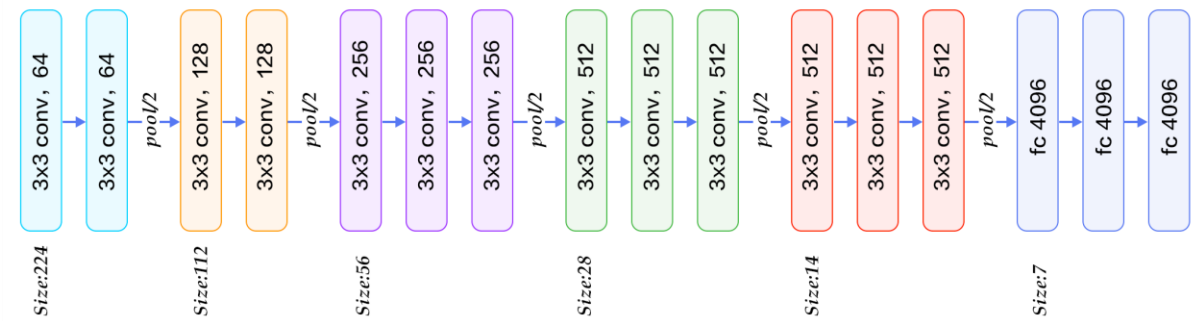
Objective:

To implement and understand the working of the VGG16 deep learning model for binary image classification using transfer learning.

Description

VGG16 is a deep convolutional neural network (CNN) developed by the Visual Geometry Group (VGG) at Oxford. It achieved top accuracy on the ImageNet challenge (ILSVRC-2014) and is widely used for transfer learning.

VGG16 consists of 16 weight layers, including 13 convolutional layers and 3 fully connected layers. It uses small 3×3 filters, ReLU activation, and max pooling to extract rich hierarchical features from images.



Key Features of VGG16

1. Pretrained Weights – The model is pretrained on the ImageNet dataset, making it highly effective for feature extraction.
2. Deep Architecture – Consists of 13 convolutional layers and 3 fully connected layers.
3. Small Receptive Fields – Uses 3×3 convolution filters, making it efficient in feature extraction.
4. Max Pooling – Reduces feature map size and improves computation efficiency.
5. Transfer Learning – Instead of training from scratch, we use pretrained weights to save computation time.
6. Fine-Tuning – Allows freezing lower layers and training only the top classification layers for specific tasks.

Procedure:

Step 1: Install Dependencies

- Ensure all required libraries are installed before running the script.

- Run the following command in the terminal:
    - pip install tensorflow matplotlib numpy

Step 2: Import Required Libraries

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Flatten
import numpy as np
import matplotlib.pyplot as plt
```

Step 3: Load the Dataset and Perform Data Preprocessing

Data Preprocessing Details

- Rescaling: Normalizes pixel values between 0 and 1.
- Validation Split: 80% training, 20% validation.
- Flow From Directory: Automatically labels images based on folder names.

```
# Define dataset directory
train_dir = 'D:/Lab/dataset/training_set'

# Apply Image Augmentation and Normalization
datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

# Load Training Data
train_generator = datagen.flow_from_directory(  train_dir, target_size=(224, 224),
batch_size=4,  class_mode='binary', subset='training')

# Load Validation Data
validation_generator = datagen.flow_from_directory( train_dir, target_size=(224, 224),
batch_size=4,  class_mode='binary', subset='validation')
```

Step 4: Load Pretrained VGG16 Model

Pretrained VGG16

- Saves computation – No need to train from scratch.
- Extracts high-level features from images efficiently.
- Pretrained on ImageNet, a large-scale dataset with millions of images.

```
# Load VGG16 Model with pretrained ImageNet weights
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze pretrained layers
for layer in base_model.layers:
    layer.trainable = False
```

Step 5: Add Custom Classification Layers

```
x = Flatten()(base_model.output)        # Flatten extracted features
x = Dense(64, activation='relu')(x)     # Fully connected layer with ReLU
x = Dense(1, activation='sigmoid')(x)   # Output layer for binary classification

# Define the final model
model = Model(inputs=base_model.input, outputs=x)
```

Step 6: Define Loss Function and Optimizer

Compilation Details

- Optimizer: Adam – Efficient adaptive learning rate optimization.
- Loss Function: binary_crossentropy – Since it's a binary classification problem.
- Metric: accuracy – Used to measure model performance.

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Step 7: Train the Model

Training Details

- Dataset is fed in batches using train_generator.
- Epochs: Set to 5 (can be increased for better performance).
- Validation Data: Helps in tracking overfitting.

```
history = model.fit(train_generator, epochs=5, validation_data=validation_generator)
```

Step 8: Evaluate the Model

```
# Evaluate the model on validation data
loss, accuracy = model.evaluate(validation_generator)
print(f"Validation Accuracy: {accuracy:.2f}")
```

Step 9: Load and Preprocess a Single Image for Prediction

```
# Load and preprocess the test image
img_path = 'D:/Lab/dataset/single_prediction/catf.jpg'
img = tf.keras.preprocessing.image.load_img(img_path, target_size=(224, 224))
img_array = np.expand_dims(tf.keras.preprocessing.image.img_to_array(img), axis=0) /
255.0
```

Step 10: Make a Prediction

Prediction Details

- Outputs a probability score (closer to 1 → Dog, closer to 0 → Cat).
- Threshold at 0.5 decides the class.

```
# Predict class
```

```
prediction = model.predict(img_array)
print(prediction)

# Assign label based on threshold
predicted_class = "Dog" if prediction[0][0] > 0.5 else "Cat"
```

Step 11: Display the Image with Prediction

```
plt.imshow(img)
plt.title(f"Prediction: {predicted_class}")
plt.axis("off")
plt.show()
```

8. Word Representations using One-Hot Encoding and Word2Vec

Objective:

To implement Word Representations Using One-Hot Encoding and Word2Vec.

Description
Computers cannot process human language directly. They require numerical representations of words to perform NLP tasks such as text classification, machine translation, and sentiment analysis.
Two common techniques for representing words in NLP are:

1. One-Hot Encoding – Simple binary vector representation.
2. Word2Vec – Dense word embeddings that capture semantic relationships.

1. One-Hot Encoding:

One-Hot Encoding represents each word as a **binary vector** of length equal to the vocabulary size.

- Each word is uniquely **assigned an index** in the vocabulary.
- A **vector is created** where only one position is **1**, and the rest are **0**.
- Each word is represented uniquely using a sparse binary vector.
- Not scalable for large vocabularies due to high-dimensionality.

Example:

Word	One-Hot Vector
AI	[0 0 1 0 0]
love	[0 1 0 0 0]
deep	[0 0 0 1 0]

Implementation:

```
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer

# Sample text
sentences = ["I love AI", "AI is powerful", "I love deep learning"]

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)

# Convert sentences to sequences
sequences = tokenizer.texts_to_sequences(sentences)
vocab_size = len(tokenizer.word_index) + 1  # +1 for padding

# One-Hot Encoding using np.eye()
def one_hot_encode(sequences, vocab_size):
    return [np.eye(vocab_size)[seq] for seq in sequences]  # Process each sentence separately
```

```
# Apply One-Hot Encoding
one_hot_vectors = one_hot_encode(sequences, vocab_size)

# Print One-Hot Encoding for the first sentence
print("\n🔹 Vocabulary Mapping:", tokenizer.word_index)
print("\n🔹 One-Hot Encoding for first sentence:")
for word, vector in zip(sentences[0].split(), one_hot_vectors[0]):
    print(f"{word} → {vector}")
```

2. Word Embeddings (Word2Vec)

Although One-Hot Encoding is simple, it has several limitations:

- High-dimensional – Requires a large number of features for big vocabularies.
- No semantic meaning – Words that are similar in meaning do not have similar vectors.

Word2Vec embeddings that represent words in a dense, meaningful vector space.
Word2Vec is a neural network-based technique that learns **word meanings** by analyzing how words appear in different contexts.

- It captures **semantic relationships**, such as:
  - "king" - "man" + "woman" ≈ "queen"
  - "Paris" - "France" + "Italy" ≈ "Rome"

Two Approaches of Word2Vec:

Model	**How it Works**
CBOW (Continuous Bag of Words)	Predicts the target word from surrounding context words.
Skip-Gram	Predicts context words given a target word.

- Word2Vec is superior for real-world NLP tasks.
- One-Hot Encoding is useful for small vocabularies and simple models.

Implementation:

```
from gensim.models import Word2Vec
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer

# Sample text corpus
sentences = ["I love AI", "AI is powerful", "I love deep learning"]

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)

# Convert sentences into lists of words for Word2Vec
tokenized_sentences = [sentence.split() for sentence in sentences]
```

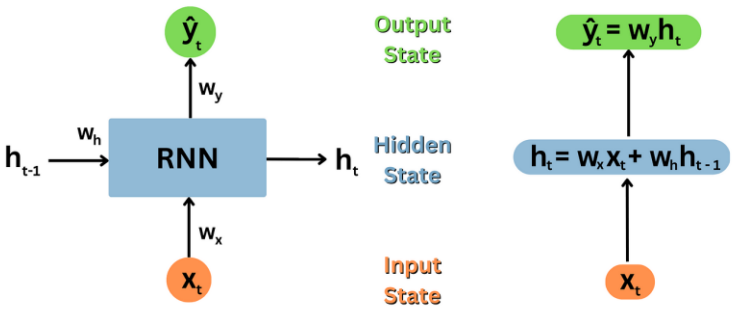
```
# Train Word2Vec model (CBOW: sg=0, Skip-Gram: sg=1)
word2vec_model = Word2Vec(sentences=tokenized_sentences, vector_size=10, window=2,
min_count=1, sg=0)

# Print Word2Vec word embeddings
print("\n🔹 Word2Vec Embedding for 'AI':")
print(word2vec_model.wv['AI'])
```

9. Recurrent Neural Networks (RNN) for Time Series Prediction

Objective:

To predict the next temperature value based on a sequence of three previous temperature readings using Simple RNN.

Architecture



Description:

A Recurrent Neural Network (RNN) is a type of artificial neural network designed to process sequential data by maintaining a memory of previous inputs. Unlike traditional feedforward neural networks, RNNs have loops that allow information to persist, making them well-suited for tasks involving time-series data, natural language processing (NLP), and speech recognition.

Characteristics of RNNs:

1. Sequential Processing: RNNs process data step by step, making them ideal for time-dependent data.
2. Memory Retention: The hidden state in RNNs stores information from previous time steps, allowing the network to capture context.
3. Shared Weights: The same set of weights is applied across all time steps, reducing the number of parameters.
4. Backpropagation Through Time (BPTT): A variation of backpropagation used to train RNNs by considering dependencies across time steps.

Recurrent Neural Network (RNN) is implemented to predict the next value in a numerical sequence. The model takes 3 time steps as input and is trained on a dataset where each sequence predicts the next number. The architecture consists of a SimpleRNN layer with 8 neurons followed by a Dense output layer. The model is compiled using the Adam optimizer and Mean Squared Error (MSE) loss function. It is trained for 100 epochs with a batch size of 2. After training, the model predicts the next number in a given sequence. The output demonstrates the RNN's ability to learn sequential dependencies. For better performance,

LSTM or GRU layers can be used instead of SimpleRNN to capture long-term dependencies. This model serves as a foundation for time-series forecasting applications.

Code:

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

# input shape: (num_samples, seq_length=3, input_dim=1)
X_train = np.array([ [20, 30, 23], [30, 23, 29], [23, 29, 28], [29, 28, 27],
[28, 27, 30], [27, 30, 32]]).reshape(6, 3, 1)  # Reshaped to 3D (samples, time
steps, features)
y_train = np.array([29, 28, 27, 30, 32, 31])  # Target values

# Define the model
model = Sequential([
    SimpleRNN(units=8, activation='relu', input_shape=(3, 1)),  # RNN Layer
    Dense(units=1)  # Output Layer
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Model Summary
model.summary()

# Train the model
model.fit(X_train, y_train, epochs=100, batch_size=2, verbose=1)  # Use
batch_size=2

# Generate test data (shape: 1 sample, 3 time steps, 1 feature)
X_test = np.array([20, 30, 23]).reshape(1, 3, 1)

# Make predictions
y_pred = model.predict(X_test)

# Print predictions
print("Predicted Output:\n", y_pred)
```

10. Time Series Forecasting Using Long Short-Term Memory (LSTM) Networks

Aim:

To predict the maximum and minimum temperature of a city using a Long Short-Term Memory (LSTM) model based on the past 30 days' temperature data.

Description:

Traditional RNNs struggle with learning long-term dependencies due to vanishing gradient problems, making it difficult to capture patterns over long sequences. LSTM addresses this issue by introducing a memory cell structure that allows information to persist over long sequences.

LSTM Architecture

An LSTM unit consists of three key gates that regulate the flow of information:

1. **Forget Gate (f_t)**: Decides what information to discard from the previous cell state.

$$f_t = \sigma(\mathrm{W}_f \cdot [\mathrm{h}_{t-1}, \mathrm{x}_t] + \mathrm{b}_f)$$

2. **Input Gate (i_t)**: Decides what new information should be stored in the cell state.

$$i_t = \sigma(\mathrm{W}_i \cdot [\mathrm{h}_{t-1}, \mathrm{x}_t] + \mathrm{b}_i)$$

3. **Cell State ($\widetilde{c_t}$)**: A candidate cell state is created using the current input and previous hidden state.

$$\widetilde{c_t} = \tanh(\mathrm{W}_c \cdot [\mathrm{h}_{t-1}, \mathrm{x}_t] + \mathrm{b}_c)$$

4. **Output Gate (o_t)**: Determines the output of the LSTM cell.

$$o_t = \sigma(\mathrm{W}_o \cdot [\mathrm{h}_{t-1}, \mathrm{x}_t] + \mathrm{b}_i)$$

5. **Cell State Update (c_t)**:

$$c_t = f_t * c_t + i_t * \widetilde{c_t}$$

6. **Hidden State Update (h_t)**:

$$h_t = o_t * \tanh(c_t)$$

Procedure:

This experiment focuses on time series forecasting of temperature using deep learning techniques. The primary steps include:

1. Data Collection: A dataset containing the maximum and minimum daily temperatures of Rajam for 30 days is created.
2. Data Preprocessing: Normalization is applied to scale the temperature values between 0 and 1 using MinMaxScaler.
3. Sequence Preparation: Data is converted into sequential input-output pairs to train the LSTM model.
4. Model Training: A neural network with an LSTM layer is built and trained using past temperature sequences.
5. Prediction: The trained model predicts the next day's temperature, and the error between the predicted and actual values is computed.

Code:

```
import numpy as np
import tensorflow as tf
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 30-day temperature data for Rajam
print("Step 1: Creating Temperature Dataset")
data = {
    "Max Temp (°C)": [38, 38, 38, 39, 39, 39, 39, 39, 39, 39, 39, 39, 39, 39, 39,
39, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40],
    "Min Temp (°C)": [22, 22, 23, 23, 24, 23, 24, 24, 23, 24, 24, 24, 24, 24, 24,
24, 24, 24, 24, 24, 24, 24, 24, 24, 24, 24, 24, 24, 24, 24]
}
df = pd.DataFrame(data)
print(df.head())

# Normalize data
print("Step 2: Normalizing Data")
scaler = MinMaxScaler()
temp_scaled = scaler.fit_transform(df)
print(temp_scaled[:5])

# Prepare sequences
print("Step 3: Preparing Sequences")
seq_length = 3
X_seq = [temp_scaled[i:i+seq_length] for i in range(len(temp_scaled) -
seq_length)]
y_seq = [temp_scaled[i+seq_length] for i in range(len(temp_scaled) - seq_length)]
X_seq, y_seq = np.array(X_seq), np.array(y_seq)
print("Sample X_seq:", X_seq[:2])
print("Sample y_seq:", y_seq[:2])

# Build & Train LSTM Model
print("Step 4: Building and Training LSTM Model")
model = tf.keras.Sequential([
```

```
    tf.keras.layers.LSTM(50, input_shape=(seq_length, 2)),
    tf.keras.layers.Dense(2)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X_seq, y_seq, epochs=20, batch_size=8, verbose=0)
print("Model training complete.")

# Single Prediction for Last Day
print("Step 5: Predicting Last Day Temperatures")
predicted_temp = scaler.inverse_transform(model.predict(X_seq[-1].reshape(1,
seq_length, 2)))
actual_temp = scaler.inverse_transform(y_seq[-1].reshape(1, -1))
error = np.abs(predicted_temp - actual_temp)

print("Predicted (Max, Min):", predicted_temp[0])
print("Actual (Max, Min):", actual_temp[0])
print("Error (Max, Min):", error[0])
```

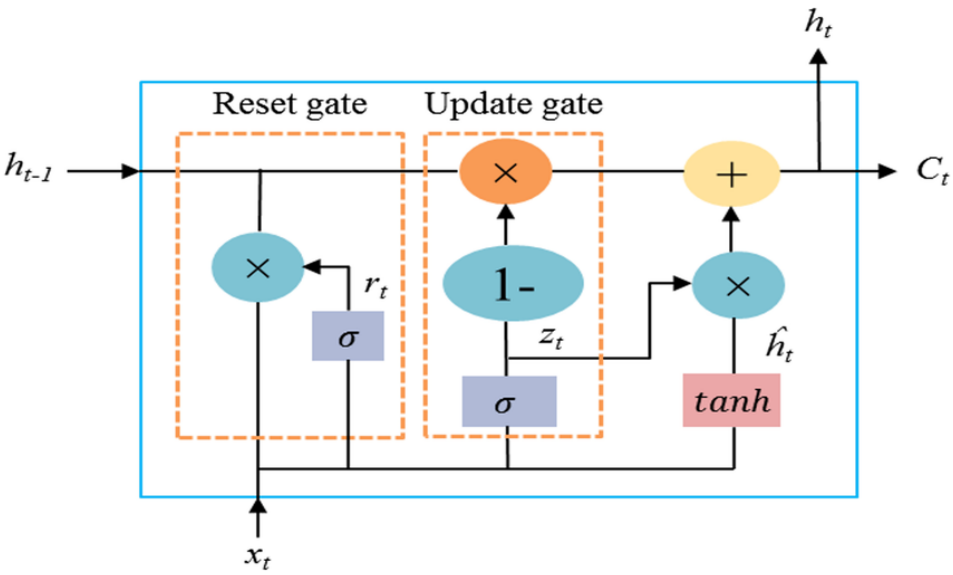
11. Stock Price Prediction Using Gated Recurrent Unit (GRU) Networks

Aim: To predict the Stock Price Prediction Using Gated Recurrent Unit (GRU) Networks based on the past few days.

Description: Recurrent Neural Networks (RNNs) are commonly used for sequence modeling, but they struggle with long-term dependencies due to the vanishing gradient problem. GRU is a type of gated RNN that overcomes this by using two gates—Reset Gate and Update Gate—making it computationally more efficient than LSTM.

GRU Architecture:

A GRU unit contains the following gates:



- **Update Gate (z_t):** Controls how much of the past information to carry forward.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

- **Reset Gate (r_t):** Determines how much of the previous memory to forget.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

- **Candidate Activation ($\tilde{h}_t$):** Combines new input with the past hidden state.

$$\tilde{h}_t = tan\,h(W_h \cdot [r_t * h_{t-1}, x_t] + b_h)$$

- **Final Output (h_t):**

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Procedure:

This experiment focuses on time series forecasting of temperature using deep learning techniques. The steps include:

1. **Data Collection:** Create a dataset containing the daily opening and closing stock prices for the past 30 days.

2. **Data Preprocessing:** Normalize the stock price values using MinMaxScaler to scale them between 0 and 1, which helps the GRU model converge faster and perform better.

3. **Sequence Preparation:** Convert the normalized data into sequences of input-output pairs, where each input is a set of stock prices over a fixed number of past days (e.g., 3), and the output is the stock prices for the next day.

4. **Model Training:** Build and train a GRU-based neural network using the prepared sequences. The model learns to map past stock prices to future prices.

5. **Prediction:** Use the trained model to predict the stock opening and closing prices for the next day based on recent historical data. Compare the predicted values with the actual values and compute the prediction error.

Code:

```
import numpy as np
import tensorflow as tf
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Step 1: Creating Stock Dataset (Dummy example for 30 days)
print("Step 1: Creating Stock Price Dataset")
data = {
    "Open Price": [100, 102, 101, 103, 105, 107, 108, 110, 109, 111, 112, 114,
113, 115, 117, 118, 119, 121, 120, 122, 123, 125, 124, 126, 127, 129, 130,
132, 131, 133],
    "Close Price": [101, 103, 102, 104, 106, 108, 109, 111, 110, 112, 113,
115, 114, 116, 118, 119, 120, 122, 121, 123, 124, 126, 125, 127, 128, 130,
131, 133, 132, 134]
}
df = pd.DataFrame(data)
print(df.head())

# Step 2: Normalizing Data
print("Step 2: Normalizing Data")
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)
print(scaled_data[:5])

# Step 3: Preparing Sequences
print("Step 3: Preparing Sequences")
seq_length = 3
X_seq = [scaled_data[i:i+seq_length] for i in range(len(scaled_data) -
seq_length)]
y_seq = [scaled_data[i+seq_length] for i in range(len(scaled_data) -
seq_length)]
X_seq, y_seq = np.array(X_seq), np.array(y_seq)
print("Sample X_seq:", X_seq[:2])
print("Sample y_seq:", y_seq[:2])
```

```python
# Step 4: Building and Training GRU Model
print("Step 4: Building and Training GRU Model")
model = tf.keras.Sequential([
    tf.keras.layers.GRU(50, input_shape=(seq_length, 2)),
    tf.keras.layers.Dense(2)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X_seq, y_seq, epochs=20, batch_size=8, verbose=0)
print("Model training complete.")

# Step 5: Predicting Next Day Stock Prices
print("Step 5: Predicting Next Day Stock Prices")
predicted_price = scaler.inverse_transform(model.predict(X_seq[-1].reshape(1,
seq_length, 2)))
actual_price = scaler.inverse_transform(y_seq[-1].reshape(1, -1))
error = np.abs(predicted_price - actual_price)

print("Predicted (Open, Close):", predicted_price[0])
print("Actual (Open, Close):", actual_price[0])
print("Error (Open, Close):", error[0])
```

Ex 13. Object Detection with YOLOv5

Objective:

To detect and count the number of persons in a given image using a pre-trained YOLOv5s object detection model.

Description:

YOLOv5 (You Only Look Once version 5) is a state-of-the-art, real-time object detection model developed by Ultralytics. It is known for its speed, accuracy, and ease of use. YOLOv5 is implemented in PyTorch and provides a flexible architecture for detecting multiple object classes within images or video streams.

Unlike earlier YOLO versions written in Darknet, YOLOv5 is natively implemented in PyTorch, making it easier to integrate with modern deep learning workflows. It supports various model sizes (yolov5s, yolov5m, yolov5l, yolov5x) to balance performance and computational efficiency.

Key features of YOLOv5 include:

- **Single-stage detection**: YOLOv5 performs object localization and classification in a single forward pass, ensuring high-speed inference.
- **Pre-trained models**: Trained on the COCO dataset, YOLOv5 can detect 80 common object categories.
- **High accuracy**: It achieves competitive mean Average Precision (mAP) scores across benchmarks.
- **Lightweight and fast**: Especially the yolov5s version, suitable for edge devices and real-time applications.

YOLOv5 is widely used in applications like surveillance, traffic monitoring, healthcare, retail analytics, and robotics due to its real-time processing capability and robustness.

Procedure:

1. **Model Loading**: A pre-trained YOLOv5s model is loaded via PyTorch Hub.
2. **Image Loading**: The input image is read using OpenCV and converted from BGR to RGB format to match the model's requirements.
3. **Inference**: The model performs inference on the input image and returns detection results.
4. **Filtering Detections**: The detection results are converted into a Pandas DataFrame and filtered to retain only the detections of class 'person'.
5. **Counting**: The number of person detections is counted and displayed.
6. **Visualization**: The annotated image with bounding boxes around detected persons is displayed using the model's built-in visualization.

Code:

```
import torch
import cv2
```

```python
# Load pre-trained YOLOv5s model
model = torch.hub.load('ultralytics/yolov5', 'yolov5s', pretrained=True)

# Load test image
img_path = 'Ex13-sample.jpg'
img = cv2.imread(img_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Inference
results = model(img_rgb)

# Get detections as pandas DataFrame
df = results.pandas().xyxy[0]

# Filter only 'person' class
person_detections = df[df['name'] == 'person']
person_count = len(person_detections)

# Print person count
print(f"Number of persons detected: {person_count}")

# Show annotated image
results.show()
```